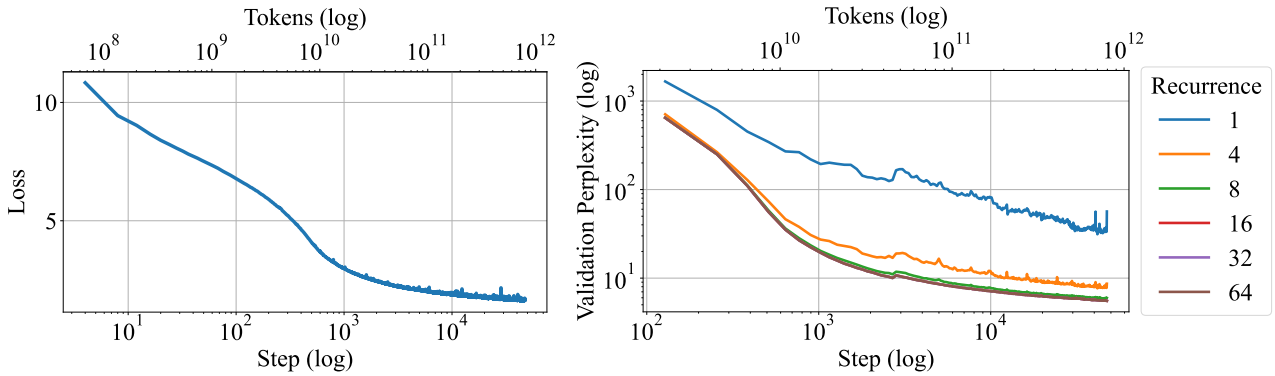

Figure 6: Left: Plot of pretrain loss over the 800B tokens on the main run. Right: Plot of val ppl at recurrent depths 1, 4, 8, 16, 32, 64. During training, the model improves in perplexity on all levels of recurrence.

Table 1: Results on lm-eval-harness tasks zero-shot across various open-source models. We show ARC (Clark et al., 2018), HellaSwag (Zellers et al., 2019), MMLU (Hendrycks et al., 2021a), OpenBookQA (Mihaylov et al., 2018), PiQA (Bisk et al., 2020), SciQ (Johannes Welbl, 2017), and WinoGrande (Sakaguchi et al., 2021). We report normalized accuracy when provided.

Model	Param	Tokens	ARC-E	ARC-C	HellaSwag	MMLU	OBQA	PiQA	SciQ	WinoGrande
random			25.0	25.0	25.0	25.0	25.0	50.0	25.0	50.0
Amber	7B	1.2T	65.70	37.20	72.54	26.77	41.00	78.73	88.50	63.22
Pythia-2.8b	2.8B	0.3T	58.00	32.51	59.17	25.05	35.40	73.29	83.60	57.85
Pythia-6.9b	6.9B	0.3T	60.48	34.64	63.32	25.74	37.20	75.79	82.90	61.40
Pythia-12b	12B	0.3T	63.22	34.64	66.72	24.01	35.40	75.84	84.40	63.06
OLMo-1B	1B	3T	57.28	30.72	63.00	24.33	36.40	75.24	78.70	59.19
OLMo-7B	7B	2.5T	68.81	40.27	75.52	28.39	42.20	80.03	88.50	67.09
OLMo-7B-0424	7B	2.05T	75.13	45.05	77.24	47.46	41.60	80.09	96.00	68.19
OLMo-7B-0724	7B	2.75T	74.28	43.43	77.76	50.18	41.60	80.69	95.70	67.17
OLMo-2-1124	7B	4T	82.79	57.42	80.50	60.56	46.20	81.18	96.40	74.74
Ours, ($r = 4$)	3.5B	0.8T	49.07	27.99	43.46	23.39	28.20	64.96	80.00	55.24
Ours, ($r = 8$)	3.5B	0.8T	65.11	35.15	58.54	25.29	35.40	73.45	92.10	55.64
Ours, ($r = 16$)	3.5B	0.8T	69.49	37.71	64.67	31.25	37.60	75.79	93.90	57.77
Ours, ($r = 32$)	3.5B	0.8T	69.91	38.23	65.21	31.38	38.80	76.22	93.50	59.43

5. Benchmark Results

We train our final model for 800B tokens, and a non-recurrent baseline for 180B tokens. We evaluate these checkpoints against other open-source models trained on fully public datasets (like ours) of a similar size. We compare against Amber (Liu et al., 2023c), Pythia (Biderman et al., 2023) and a number of OLMo 1&2 variants (Groeneveld et al., 2024; AI2, 2024; Team OLMo et al., 2025). We execute all standard benchmarks through the lm-eval harness (Biderman et al., 2024) and code benchmarks via bigcode-bench (Zhuo et al., 2024).

5.1. Standard Benchmarks

Overall, it is not straightforward to place our model in direct comparison to other large language models, all of which are small variations of the fixed-depth transformer architecture. While our model has only 3.5B parameters and hence requires only modest interconnect bandwidth during pretraining, it chews through raw FLOPs close to what a 32B parameter transformer would consume during pretraining, and can continuously improve in performance with test-time scaling up to FLOP budgets equivalent to a standard 50B parameter fixed-depth transformer. It is also important to note a few caveats of the main training run when interpreting the results. First, our main checkpoint is trained for only 47000 steps on a broadly untested mixture, and the learning rate is never cooled down from its peak. As an academic project, the model is trained only on publicly available data and the 800B token count, while large in comparison to older fully open-source models such as the Pythia series, is small in comparison to modern open-source efforts such as OLMo, and tiny in comparison to the datasets used to train industrial open-weight models.

Disclaimers aside, we collect results for established benchmark tasks (Team OLMo et al., 2025) in Table 1 and show all models side-by-side. In direct comparison we see that our model outperforms the older Pythia series and is roughly comparable to the first OLMo generation, OLMo-7B in most metrics, but lags behind the later OLMo models trained larger, more carefully curated datasets. For the first recurrent-depth model for language to be trained at this

Table 2: Benchmarks of mathematical reasoning and understanding. We report flexible and strict extract for GSM8K and GSM8K CoT, extract match for Minerva Math, and acc norm. for MathQA.

Model	GSM8K	GSM8k CoT	Minerva MATH	MathQA
Random	0.00	0.00	0.00	20.00
Amber	3.94/4.32	3.34/5.16	1.94	25.26
Pythia-2.8b	1.59/2.12	1.90/2.81	1.96	24.52
Pythia-6.9b	2.05/2.43	2.81/2.88	1.38	25.96
Pythia-12b	3.49/4.62	3.34/4.62	2.56	25.80
OLMo-1B	1.82/2.27	1.59/2.58	1.60	23.38
OLMo-7B	4.02/4.09	6.07/7.28	2.12	25.26
OLMo-7B-0424	27.07/27.29	26.23/26.23	5.56	28.48
OLMo-7B-0724	28.66/28.73	28.89/28.89	5.62	27.84
OLMo-2-1124-7B	66.72/66.79	61.94/66.19	19.08	37.59
Our w/o sys. prompt ($r = 32$)	28.05/28.20	32.60/34.57	12.58	26.60
Our w/ sys. prompt ($r = 32$)	24.87/38.13	34.80/42.08	11.24	27.97

Table 3: Evaluation on code benchmarks, MBPP and HumanEval. We report pass@1 for both datasets.

Model	Param	Tokens	MBPP	HumanEval
Random			0.00	0.00
starcoder2-3b	3B	3.3T	43.00	31.09
starcoder2-7b	7B	3.7T	43.80	31.70
Amber	7B	1.2T	19.60	13.41
Pythia-2.8b	2.8B	0.3T	6.70	7.92
Pythia-6.9b	6.9B	0.3T	7.92	5.60
Pythia-12b	12B	0.3T	5.60	9.14
OLMo-1B	1B	3T	0.00	4.87
OLMo-7B	7B	2.5T	15.6	12.80
OLMo-7B-0424	7B	2.05T	21.20	16.46
OLMo-7B-0724	7B	2.75T	25.60	20.12
OLMo-2-1124-7B	7B	4T	21.80	10.36
Ours ($r = 32$)	3.5B	0.8T	24.80	23.17

scale, and considering the limitations of the training run, we find these results promising and certainly suggestive that further research into latent recurrence as an approach to test-time scaling is warranted.

5.2. Math and Coding Benchmarks

We also evaluate the model on math and coding. For math, we evaluate GSM8k (Cobbe et al., 2021) (as zero-shot and in the 8-way CoT setup), MATH ((Hendrycks et al., 2021b) with the Minerva evaluation rules (Lewkowycz et al., 2022)) and MathQA (Amini et al., 2019). For coding, we check MBPP (Austin et al., 2021) and HumanEval (Chen et al., 2021). Here we find that our model significantly surpasses all models except the latest OLMo-2 model in mathematical reasoning, as measured on GSM8k and MATH. On coding benchmarks the model beats all other general-purpose open-source models, although it does not outperform dedicated code models, such as StarCoder2 (Lozhkov et al., 2024), trained for several trillion tokens. We also note that while further improvements in language modeling are slowing down, as expected at this training scale, both code and mathematical reasoning continue to improve steadily throughout training, see Figure 8.

5.3. Where does recurrence help most?

How much of this performance can we attribute to recurrence, and how much to other factors, such as dataset, tokenization and architectural choices? In Table 4, we compare our recurrent model against its non-recurrent twin, which we trained to 180B tokens in the exact same setting. In direct comparison of both models at 180B tokens, we see that the recurrent model outperforms its baseline with an especially pronounced advantage on harder tasks, such as the ARC challenge set. On other tasks, such as SciQ, which requires straightforward recall of scientific facts, performance of the models is more similar. We observe that gains through reasoning are especially prominent on GSM8k, where the 180B recurrent model is already 5 times better than the baseline at this early snapshot in the pretraining

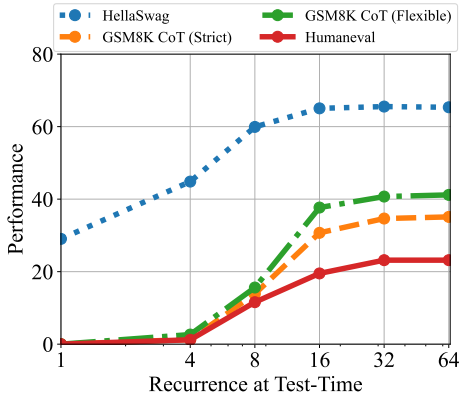

Figure 7: Performance on GSM8K CoT (strict match and flexible match), HellaSwag (acc norm.), and HumanEval (pass@1). As we increase compute, the performance on these benchmarks increases. HellaSwag only needs 8 recurrences to achieve near peak performance while other benchmarks make use of more compute.

process. We also note that the recurrent model, when evaluated with only a single recurrence, effectively stops improving between the early 180B checkpoint and the 800B checkpoint, showing that further improvements are not built into the prelude or coda non-recurrent layers but encoded entirely into the iterations of the recurrent block.

Further, we chart the improvement as a function of test-time compute on several of these tasks for the main model in Figure 7. We find that saturation is highly task-dependent, on easier tasks the model saturates quicker, whereas it benefits from more compute on others.

Recurrence and Context We evaluate ARC-C performance as a function of recurrence and number of few-shot examples in the context in Figure 9. Interestingly, without few-shot examples to consider, the model saturates in compute around 8-12 iterations. However, when more context is given, the model can reason about more information in context, which it does, saturating around 20 iterations if 1 example is provided, and 32 iterations, if 25-50 examples are provided, mirroring generalization improvements shown for recurrence (Yang et al., 2024a; Fan et al., 2025). Similarly,

Table 4: Baseline comparison, recurrent versus non-recurrent model trained in the same training setup and data. Comparing the recurrent model with its non-recurrent baseline, we see that even at 180B tokens, the recurrent substantially outperforms on harder tasks.

Model	Tokens	ARC-E	ARC-C	HellaSwag	MMLU	OBQA	PiQA	SciQ	WinoGrande	GSM8K CoT
Fixed-Depth Baseline	0.18T	46.42	26.96	37.34	24.16	29.60	64.47	73.20	51.78	1.82/2.20
Ours, early ckpt, ($r = 32$)	0.18T	53.62	29.18	48.80	25.59	31.40	68.88	80.60	52.88	9.02/10.24
Ours, early ckpt, ($r = 1$)	0.18T	34.01	23.72	29.19	23.47	25.60	53.26	54.10	53.75	0.00/0.15
Ours, ($r = 32$)	0.8T	69.91	38.23	65.21	31.38	38.80	76.22	93.50	59.43	34.80/42.08
Ours, ($r = 1$)	0.8T	34.89	24.06	29.34	23.60	26.80	55.33	47.10	49.41	0.00/0.00

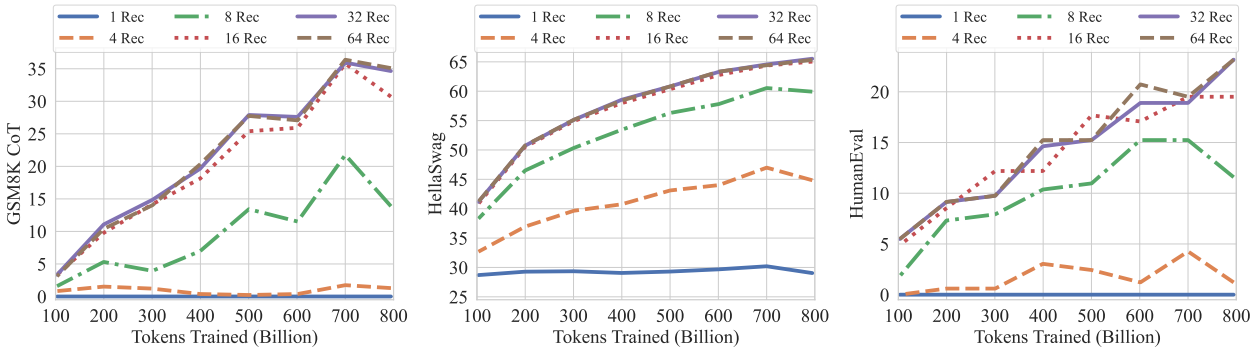

Figure 8: GSM8K CoT, HellaSwag, and HumanEval performance over the training tokens with different recurrences at test-time. We evaluate GSM8K CoT with chat template and 8-way few shot as multiturn. HellaSwag and HumanEval are zero-shot with no chat template. Model performance on harder tasks grows almost linearly with the training budget, if provided sufficient test-time compute.

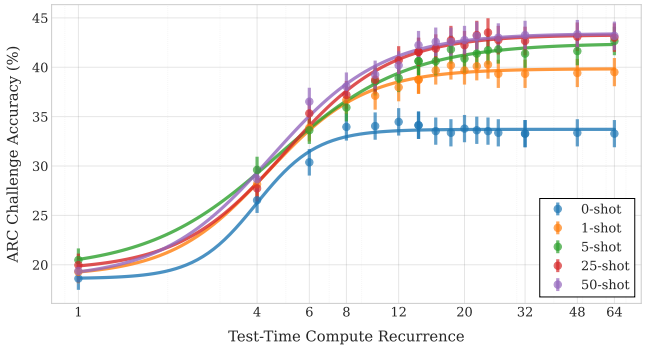

Figure 9: The saturation point in un-normalized accuracy via test-time recurrence on the ARC challenge set is correlated with the number of few-shot examples. The model uses more recurrence to extract more information from the additional few-shot examples, making use of more compute if more context is given.

Table 5: Comparison of Open and Closed QA Performance (%) (Mihaylov et al., 2018). In the open exam, a relevant fact is provided before the question is asked. In this setting, our smaller model closes the gap to other open-source models, indicating that the model is capable, but has fewer facts memorized.

Model	**Closed**	**Open**	**Δ**
Amber	41.0	46.0	+5.0
Pythia-2.8b	35.4	44.8	+9.4
Pythia-6.9b	37.2	44.2	+7.0
Pythia-12b	35.4	48.0	+12.6
OLMo-1B	36.4	43.6	+7.2
OLMo-7B	42.2	49.8	+7.6
OLMo-7B-0424	41.6	50.6	+9.0
OLMo-7B-0724	41.6	53.2	+11.6
OLMo-2-1124	46.2	53.4	+7.2
Ours ($r = 32$)	38.2	49.2	+11.0

we see that if we re-evaluate OBQA in Table 5, but do not run the benchmark in the default lm-eval "closed-book" format and rather provide a relevant fact, our recurrent model improves significantly almost closing the gap to OLMo-2. Intuitively this makes sense, as the recurrent models has less capacity to memorize facts but more capacity to reason about its context.

5.4. Improvements through Weight Averaging

Due to our constant learning rate, we can materialize further improvements through weight averaging (Izmailov et al., 2018) to simulate the result of a cooldown (Hägele et al., 2024; DeepSeek-AI et al., 2024). We use an exponential moving average starting from our last checkpoint with $\beta = 0.9$, incorporating the last 75 checkpoints with a dilation factor of 7, a modification to established protocols (Kaddour, 2022; Sanyal et al., 2024). We provide this EMA model as well, which further improves GMS8k performance to 47.23% flexible (38.59% strict), when tested at $r = 64$.

6. Recurrent Depth simplifies LLMs

Aside from encouraging performance in mathematical and code reasoning, recurrent-depth models turn out to be surprisingly natural tools to support a number of methods that require substantial effort with standard transformers. In the next section, we provide a non-exhaustive overview.